PARQUEADERO INTELIGENTE ESP32

Documentación Técnica, Especificaciones de Hardware, Software y Manual de Operación

1. DESCRIPCIÓN DEL PROYECTO

Nombre del proyecto: PARQUEADERO INTELIGENTE - ESP32 (System Bulls).

Objetivo General: Diseñar, implementar y documentar un sistema embebido de gestión de estacionamientos totalmente automatizado e independiente sobre el microcontrolador ESP32. El sistema controla accesos vehiculares mediante RFID, detecta presencia física por magnetometría, gestiona tarifas dinámicas por tiempo consumido, opera una barrera física con servomotor y sirve interfaces web locales de administración y consulta cliente sin depender de servidores externos o internet.

Funcionamiento General:

- **Control de Acceso (RFID RC522):** El usuario aproxima una tarjeta RFID al lector. Si el vehículo está fuera y hay plazas libres, se registra su hora de entrada, se muestra un saludo personalizado en la pantalla LCD y se levanta la barrera electromecánica durante 4 segundos. Si el vehículo ya está dentro, el sistema calcula la estadía en minutos, procesa el cobro según la tarifa configurada, emite el ticket digital/pantalla y abre la barrera para dar salida. Si el aforo está al 100%, la pantalla notifica "LLENO" y deniega el acceso. Si la tarjeta no está registrada, la registra como desconocida y permite asociarla desde el panel web.
- **Monitoreo de Plaza (GY-271):** Un sensor magnético triaxial supervisa el puesto 1. Al colocarse un objeto metálico o vehículo sobre él, detecta la alteración del campo magnético terrestre (desviación magnética), aplicando algoritmos de media móvil e histéresis para determinar el estado de ocupación (LIBRE/OCUPADO).
- **Servidor Web Embebido (HTTP / REST API):** El ESP32 ejecuta simultáneamente un servidor HTTP que ofrece dos interfaces web diferenciadas en la memoria Flash (PROGMEM): un Panel de Administración (`/admin`) protegido por credenciales BASIC AUTH para el control técnico y una Web App Progresiva (PWA) de Cliente (`/`) para consultar tiempo y costos en vivo desde dispositivos móviles.

Características Principales:

- Arquitectura 100% autónoma (Edge Computing): No requiere conexión a la nube ni bases de datos externas.
- Persistencia de datos en memoria no volátil: Utiliza LittleFS y Preferences para preservar usuarios, historial y configuraciones ante cortes de energía.
- Monitoreo de campo magnético vectorial con calibración automática del puesto vacío.
- Sincronización horaria híbrida (NTP automático con conexión WiFi o sincronización por dispositivo del usuario al abrir la Web App).
- Red WiFi Dual: Funciona en modo Estación (STA) unida a un router local y/o en modo Punto de Acceso (AP) emitiendo su propia red WiFi.

Aplicaciones: Estacionamientos de centros comerciales, edificios residenciales, universidades, parqueaderos públicos automatizados y maquetas de simulación tecnológica para ingeniería y robótica.

2. COMPONENTES Y MATERIALES

COMPONENTE	CARACTERÍSTICAS TÉCNICAS	FUNCIÓN EN EL PROYECTO	ESPECIFICACIONES CLAVE
	SoC Dual-Core Tensilica Xtensa LX6 a 240 MHz, 520 KB SRAM, 4	Unidad Central de Procesamiento (MCU),	Lógica a 3.3 V. Alimentado

4. CÓDIGO DE PROGRAMACIÓN

El firmware se divide modularmente en el archivo fuente principal `parqueadero_inteligente.ino` y dos encabezados HTML/JS encapsulados en memoria PROGMEM (`web_admin.h` y `web_app.h`).

Estructura del Código Fuente Principal (Extracto de Lógica Base):

```
// Fragmento de Lógica del Servidor REST y Gestión de Accesos (parqueadero_inteligente.ino)
#include <WiFi.h> #include <WebServer.h> #include <LiquidCrystal.h> #include <MFRC522.h> #include <ESP32Servo.h> #include <ArduinoJson.h> // Inicialización de Periféricos y Variables de Configuración LiquidCrystal lcd(27, 26, 25, 33, 32, 14); MFRC522 rfid(5, 4); Servo barrera; WebServer server(80); void procesarTarjeta(const String& uid) { int i = buscarUid(uid); if (i < 0) { tarjetaDesconocida = uid; lcdMensaje("Tarjeta no", "registrada", 3000); return; } Cliente& c = clientes[i]; if (!c.dentro) { if (cuposLibres() == 0) { lcdMensaje("Parqueadero", "LLENO", 3000); return; } c.dentro = true; c.entrada = ahora(); guardarClientes(); lcdMensaje("Bienvenido", c.nombre, 4000); abrirBarrera(); } else { registrarSalida(c, true); } }
```

Explicación Funcional del Código por Secciones:

- **Definición de Constantes y Objetos:** Mapeo de pines GPIO, inicialización de estructuras de datos (`Cliente`, `Registro`) y vectores dinámicos (`std::vector`).
- **Módulo del Magnetómetro (GY-271):** La función `magIniciar()` escanea dinámicamente las direcciones I2C 0x0D (QMC5883L), 0x2C (QMC5883P) y 0x1E (HMC5883L). La función `atenderSensor()` calcula la distancia euclidiana tridimensional $D = \sqrt{(x - x_0)^2 + (y - y_0)^2 + (z - z_0)^2}$ y aplica un filtro de promedio ponderado ($D_{filtrado} = 0.7 \cdot D_{prev} + 0.3 \cdot D_{actual}$) e histéresis (umbral al 60% para liberar) garantizando lecturas estables.
- **Persistencia de Datos (LittleFS & Preferences):** `guardarClientes()` e `historial.json` serializan los vectores a formato JSON para ser almacenados en la memoria flash interna. Los parámetros operativos (tarifa, cupos, umbral magnético) se gestionan mediante la librería `Preferences.h` en la partición NVS.
- **API REST HTTP:** Implementa endpoints JSON como `/api/admin/estado`, `/api/admin/cliente`, `/api/cliente` y `/api/hora` permitiendo a las aplicaciones frontend comunicarse de manera asíncrona mediante AJAX Fetch API.

5. PROCEDIMIENTO DE CONSTRUCCIÓN

1. **Ensamblaje del Hardware:** Conectar la pantalla LCD 1602A al ESP32 siguiendo el bus de 4 bits y soldar la resistencia de contraste de 1 kΩ entre el Pin 3 (V_0) y GND. Cablear el módulo RFID RC522 respetando la tensión de 3.3 V. Conectar el sensor GY-271 a las líneas I2C (GPIO 21 y 22). Conectar la señal del servo SG90 al GPIO 13 y colocar el condensador de desacoplo de 470 µF en los rieles de alimentación 5 V / GND.
2. **Configuración del Entorno de Desarrollo:** Instalar el IDE de Arduino (v2.x). Agregar la URL de placas ESP32 en preferencias e instalar el paquete *esp32 por Espressif Systems* (versión 3.x). Instalar las bibliotecas requeridas: `MFRC522`, `ESP32Servo`, `ArduinoJson` (v7) y `LiquidCrystal`.
3. **Compilación y Carga del Firmware:** Abrir `parqueadero_inteligente.ino` (asegurando que `web_admin.h` y `web_app.h` residan en la misma carpeta). Seleccionar la tarjeta **ESP32 Dev Module**. Ajustar la velocidad del puerto serie a 115200 baudios. Cargar el programa asegurando que la plaza del Puesto 1 se encuentre totalmente desocupada durante el primer arranque para la autocalibración del sensor magnético.
4. **Pruebas de Funcionamiento y Verificación:** Abrir el Monitor Serie para verificar la inicialización del RC522 y GY-271. Conectarse a la red WiFi `Parqueadero-ESP32` (clave: `parqueo123`) e ingresar desde el navegador a `http://192.168.4.1/admin`. Pasar una tarjeta RFID, verificar la recepción del UID en el panel, registrar el cliente y comprobar la apertura mecánica de la barrera por el servo.